

Laboratorul I

Tema: Utilizarea diferitor algoritmi de clusterizare pe diferite tipuri de date. Vizualizarea datelor.

Efectuat: Perebinos Daniel

Instalarea dependentelor

```
!pip install numpy pandas matplotlib seaborn scikit-learn
```

```
Requirement already satisfied: numpy in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (2.2.0)
Requirement already satisfied: pandas in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (2.2.3)
Requirement already satisfied: matplotlib in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (3.10.0)
Requirement already satisfied: seaborn in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (0.13.2)
Requirement already satisfied: scikit-learn in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (1.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from pandas) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from pandas) (2024.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (1.4.7)
Requirement already satisfied: packaging>=20.0 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (24.2)
```

Requirement already satisfied: pillow>=8 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (11.0.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from matplotlib) (3.2.0)

Requirement already satisfied: scipy>=1.6.0 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from scikit-learn) (1.14.1)

Requirement already satisfied: joblib>=1.2.0 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from scikit-learn) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from scikit-learn) (3.5.0)

Requirement already satisfied: six>=1.5 in c:\users\daniel.perebinos\pycharmprojects\mldm_lab1\.venv\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Data preprocessing

```
import pandas as pd
```

```
df = pd.read_csv('dataset.csv', index_col=0)
df.head()
```

	SeniorCitizen	Partner	Dependents	tenure	PhoneService
0	0	1	0	1	0
1					
1	0	0	0	34	1
0					
2	0	0	0	2	1
0					
3	0	0	0	45	0
1					
4	0	0	0	2	1
0					

	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	\
0	0	0	2	0	
1	0	2	0	2	
2	0	2	2	0	
3	0	2	0	2	
4	1	0	0	0	

	TechSupport	StreamingTV	StreamingMovies	Contract
PaperlessBilling	\			

0	0	0	0	0
1				
1	0	0	0	1
0				
2	0	0	0	0
1				
3	2	0	0	1
0				
4	0	0	0	0
1				

	PaymentMethod	MonthlyCharges	Churn
0	2	29.85	0
1	3	56.95	0
2	3	53.85	1
3	0	42.30	0
4	2	70.70	1

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 7043 entries, 0 to 7042
```

```
Data columns (total 18 columns):
```

#	Column	Non-Null	Count	Dtype
0	SeniorCitizen	7043	non-null	int64
1	Partner	7043	non-null	int64
2	Dependents	7043	non-null	int64
3	tenure	7043	non-null	int64
4	PhoneService	7043	non-null	int64
5	MultipleLines	7043	non-null	int64
6	InternetService	7043	non-null	int64
7	OnlineSecurity	7043	non-null	int64
8	OnlineBackup	7043	non-null	int64
9	DeviceProtection	7043	non-null	int64
10	TechSupport	7043	non-null	int64
11	StreamingTV	7043	non-null	int64
12	StreamingMovies	7043	non-null	int64
13	Contract	7043	non-null	int64
14	PaperlessBilling	7043	non-null	int64
15	PaymentMethod	7043	non-null	int64
16	MonthlyCharges	7043	non-null	float64
17	Churn	7043	non-null	int64

```
dtypes: float64(1), int64(17)
```

```
memory usage: 1.0 MB
```

```
df.isna().sum()
```

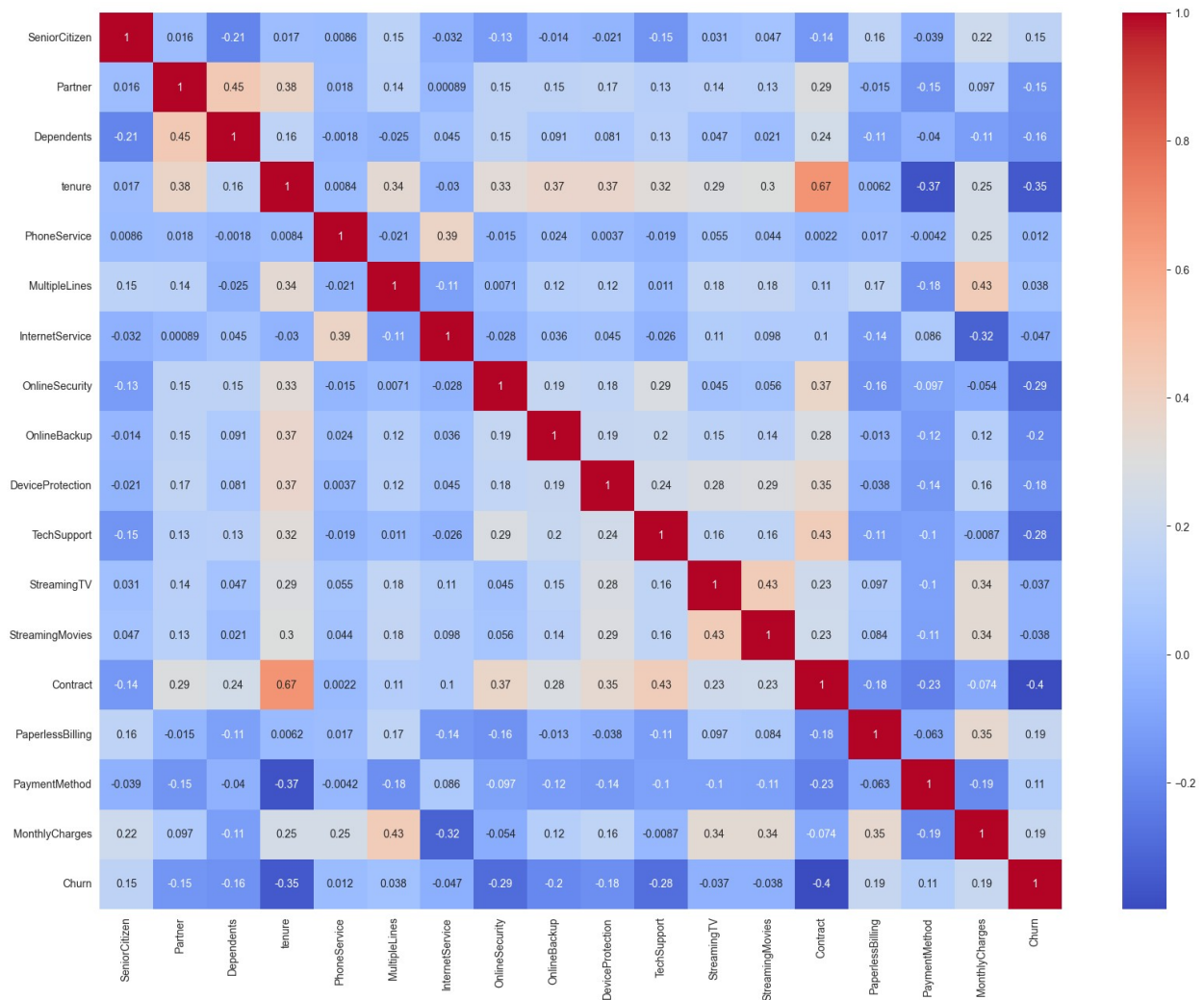
SeniorCitizen	0
Partner	0

```
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
Churn           0
dtype: int64
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(20, 15))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

```
<Axes: >
```



```
df = df.drop(columns=["Contract", "Churn"])

from sklearn.decomposition import PCA

scaler = PCA(n_components=2)
X = scaler.fit_transform(df)
```

Algoritmi partiționali (KMeans)

Principiul de lucru al KMeans constă în împărțirea datelor în k cluster pe baza distanței minime față de centrele (centrozii) clusterelor. Algoritmul ajustează centrele iterativ până când acestea converg, minimizând suma pătratelor distanțelor dintre puncte și centroizi.

```
from sklearn.cluster import KMeans

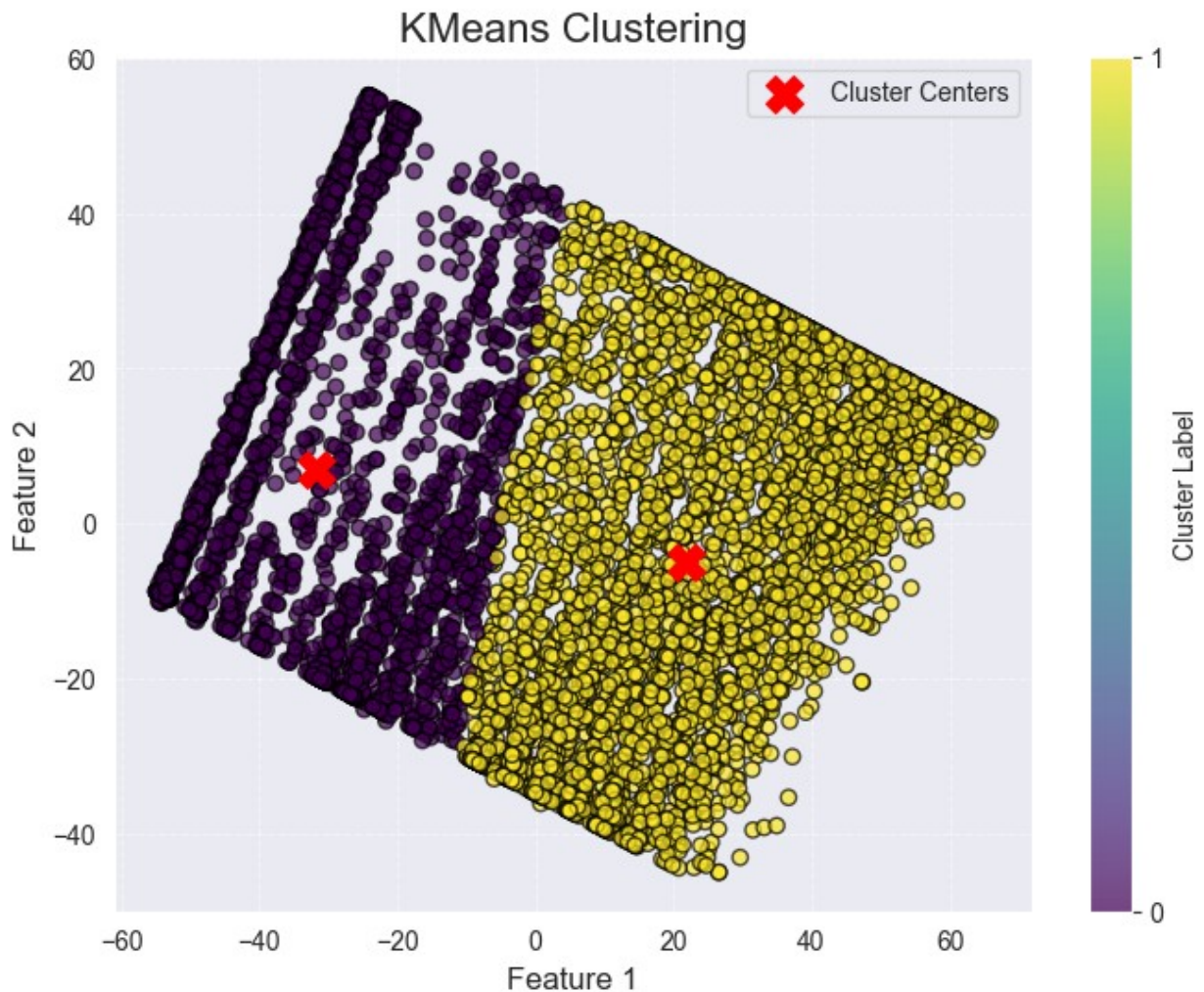
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)
```

```

labels = kmeans.labels_
centers = kmeans.cluster_centers_

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
alpha=0.7, edgecolor='k')
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='X',
label='Cluster Centers')
plt.title('KMeans Clustering', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
cbar = plt.colorbar(scatter, label='Cluster Label')
cbar.set_ticks(range(kmeans.n_clusters))
plt.legend(loc='upper right', fontsize=10)
plt.grid(visible=True, linestyle='--', alpha=0.5)
plt.show()

```



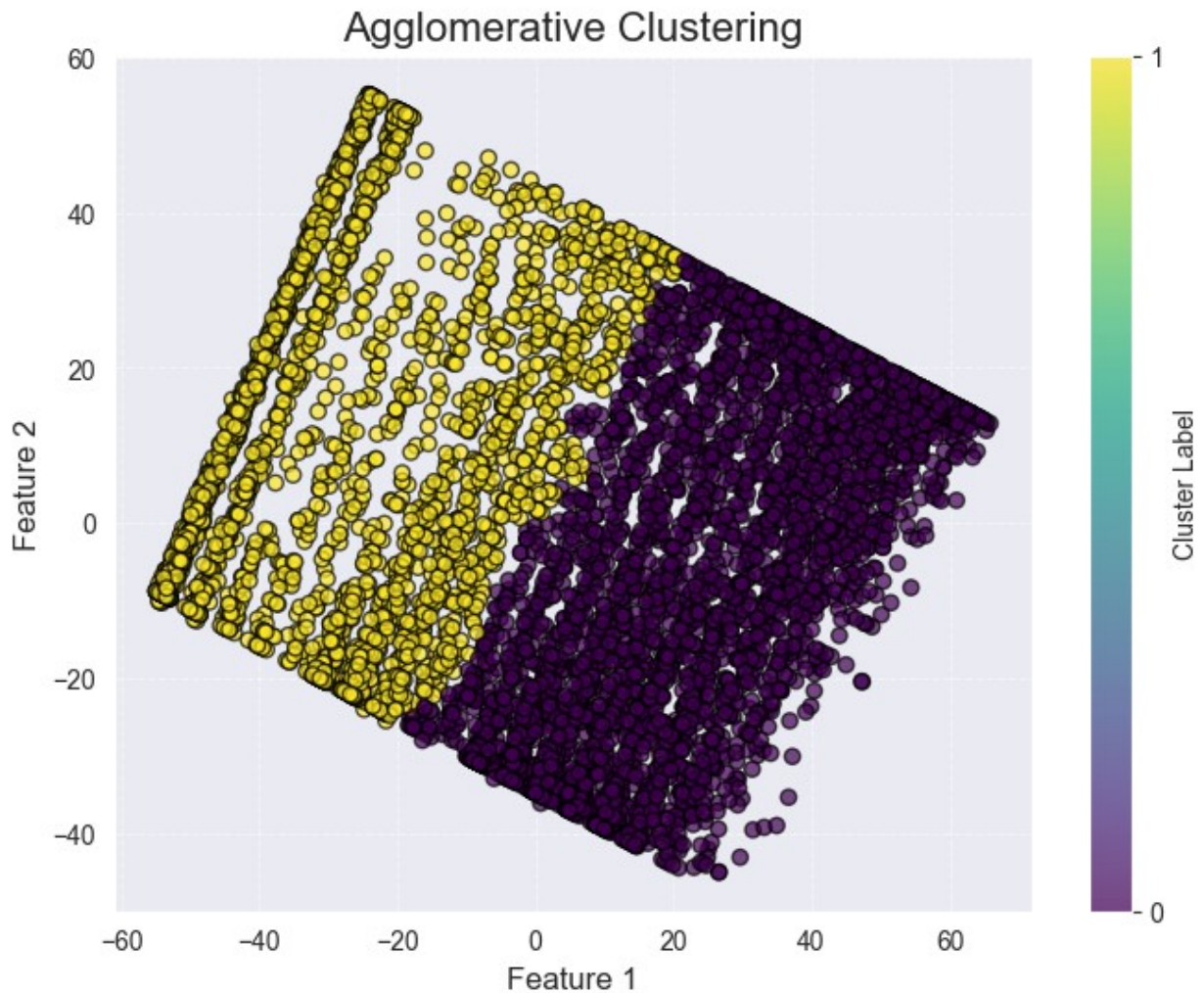
```
from sklearn.metrics import silhouette_score  
  
score = silhouette_score(X, labels)  
print(f"Silhouette Score pentru KMeans: {score:.2f}")
```

Silhouette Score pentru KMeans: 0.43

Algoritmi hierarhici (AgglomerativeClustering)

Algoritmul aglomerativ începe prin considerarea fiecărui punct ca un cluster separat, apoi combină succesiv clusterelor cele mai apropiate. Procesul continuă până când toate punctele formează un singur cluster sau este atins un număr specificat de cluster.

```
from sklearn.cluster import AgglomerativeClustering  
  
agg_clustering = AgglomerativeClustering(n_clusters=2)  
labels = agg_clustering.fit_predict(X)  
  
plt.figure(figsize=(8, 6))  
scatter = plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',  
alpha=0.7, edgecolor='k')  
plt.title('Agglomerative Clustering', fontsize=16)  
plt.xlabel('Feature 1', fontsize=12)  
plt.ylabel('Feature 2', fontsize=12)  
cbar = plt.colorbar(scatter, label='Cluster Label')  
cbar.set_ticks(range(agg_clustering.n_clusters))  
plt.grid(visible=True, linestyle='--', alpha=0.5)  
plt.show()
```

```
score = silhouette_score(X, labels)
print(f"Silhouette Score pentru AgglomerativeClustering: {score:.2f}")
```

Silhouette Score pentru AgglomerativeClustering: 0.42

Algoritmi bazați pe densitate (DBSCAN)

DBSCAN identifică clusterelor pe baza densității punctelor, grupând punctele apropiate și marcând cele izolate drept zgomot. Algoritmul nu necesită specificarea numărului de clusterelor, ci depinde de distanța minimă dintre puncte și numărul minim de vecini.

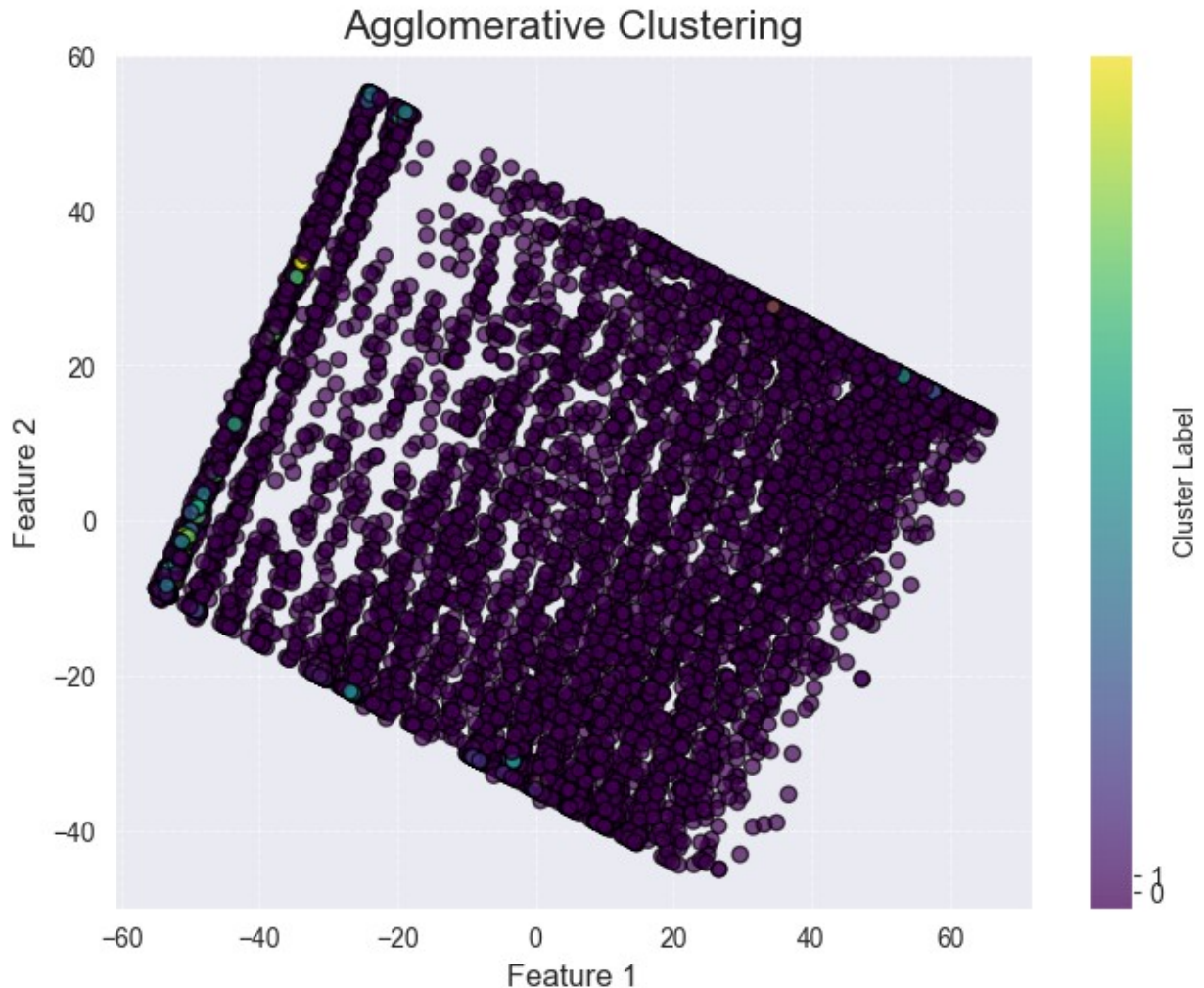
```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.2, min_samples=7)
labels = dbscan.fit_predict(X)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
```



```
alpha=0.7, edgecolor='k')
plt.title('Agglomerative Clustering', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
cbar = plt.colorbar(scatter, label='Cluster Label')
cbar.set_ticks(range(agg_clustering.n_clusters))
plt.grid(visible=True, linestyle='--', alpha=0.5)
plt.show()
```



```
score = silhouette_score(X, labels)
print(f"Silhouette Score pentru DBSCAN: {score:.2f}")
```

Silhouette Score pentru DBSCAN: -0.48

Algoritmi bazați pe modele probabiliste (GaussianMixture)

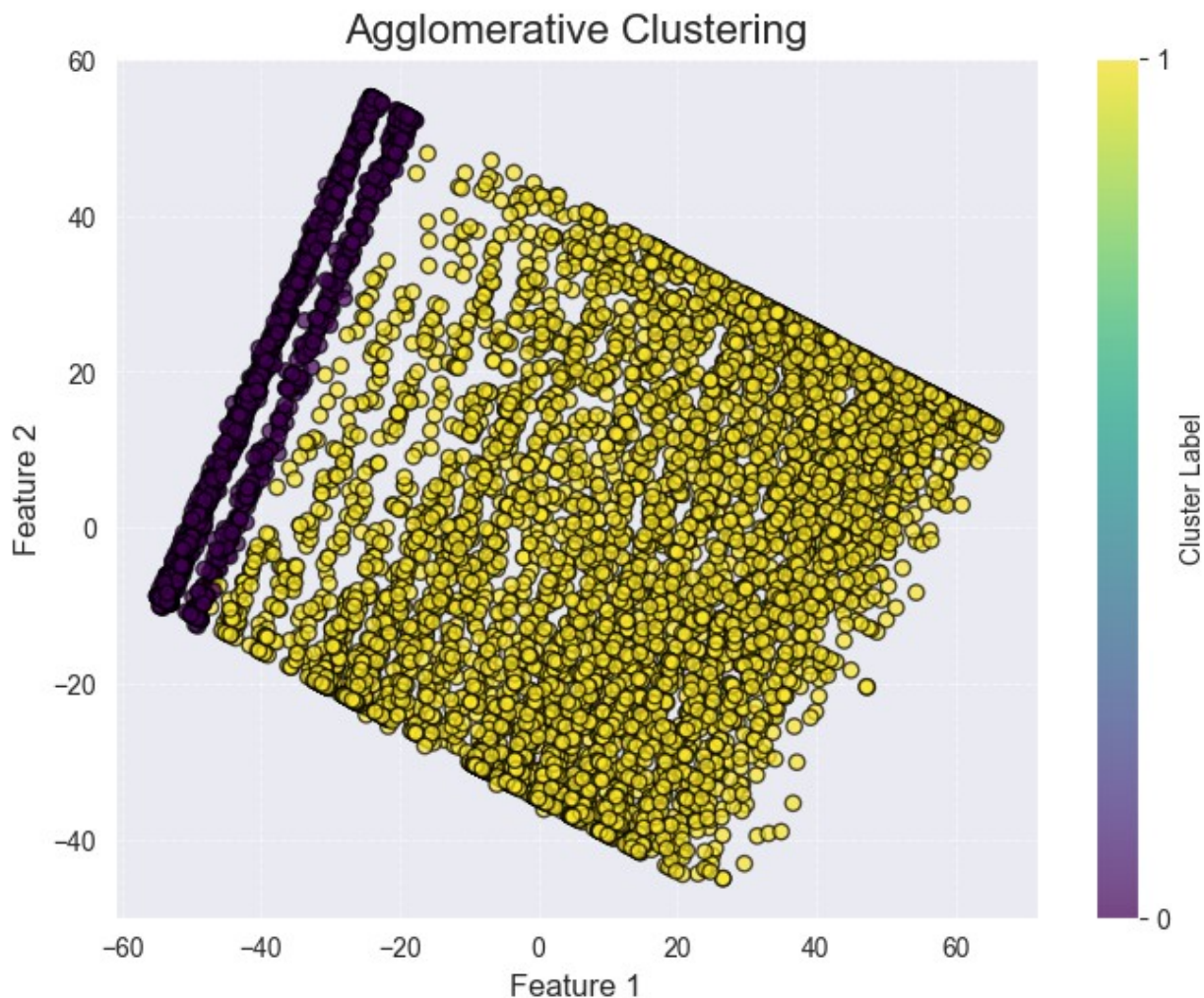
GaussianMixture presupune că datele sunt generate din mai multe distribuții Gaussiane și utilizează metoda Expectation-Maximization (EM) pentru a estima parametrii acestor distribuții.

Fiecare punct primește o probabilitate de apartenență la un cluster, iar estimările se ajustează iterativ până la convergență.

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=2, random_state=42)
gmm.fit(X)
labels = gmm.predict(X)
df["cluster"] = labels

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
alpha=0.7, edgecolor='k')
plt.title('Agglomerative Clustering', fontsize=16)
plt.xlabel('Feature 1', fontsize=12)
plt.ylabel('Feature 2', fontsize=12)
cbar = plt.colorbar(scatter, label='Cluster Label')
cbar.set_ticks(range(agg_clustering.n_clusters))
plt.grid(visible=True, linestyle='--', alpha=0.5)
plt.show()
```



Concluzie

În acest laborator, am explorat patru tipuri de algoritmi de clusterizare: KMeans, AgglomerativeClustering, DBSCAN și GaussianMixture. Fiecare algoritm a fost aplicat pe seturi de date generate, evidențiind modul în care funcționează în funcție de forma și structura datelor. Am observat că KMeans și GaussianMixture funcționează bine pentru date cu clustere bine separate, în timp ce DBSCAN este mai eficient pentru date cu forme neregulate și poate identifica zgometul. AgglomerativeClustering a demonstrat o abordare flexibilă pentru datele ierarhice. Evaluarea cu Silhouette Score a confirmat performanța fiecărui algoritm, subliniind importanța alegerii metodei potrivite în funcție de caracteristicile datelor.